

Higher-Order Programming

Concepts of Programming Languages

But first,

Algebraic Data Types

Recursive Data

```
type t
  = A
  | B of t
  | C of t * t
```

```
let x : t = B (C (A, B A))
```

A variant type **t** can carry data of type **t**

Why would we want to do this?

Example: Lists

```
type intlist  
  = Nil  
  | Cons of int * intlist
```

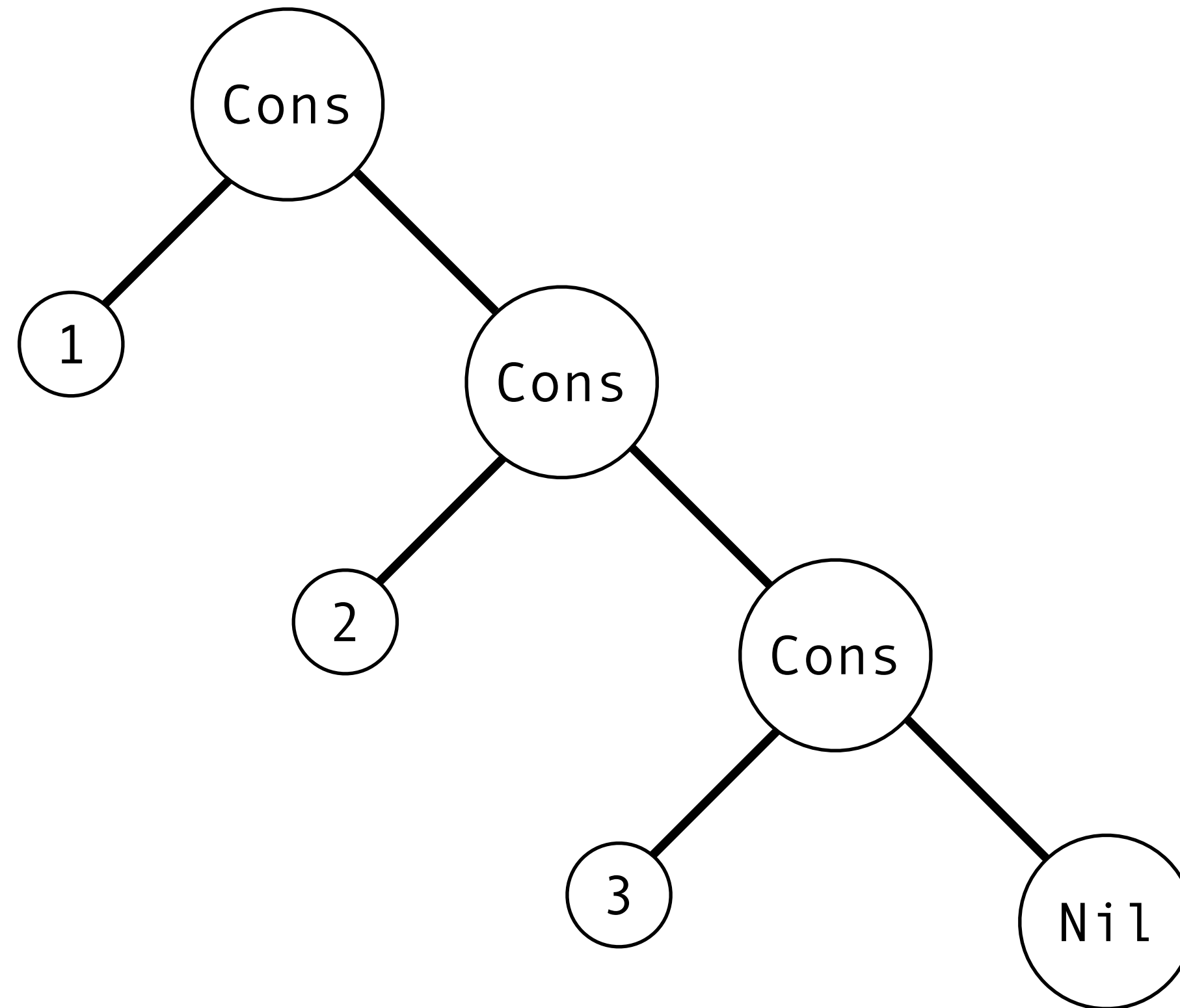
```
let example = Cons (1, Cons (2, Cons (3, Nil)))
```

The type **intlist** is available as the type of data which a constructor of **intlist** holds

We can use recursive ADTs to create variable-length data types

The Picture

```
Cons (1,  
      Cons (2,  
            Cons (3,  
                  Nil)))
```



We think of values of recursive variants as **trees** with constructors as nodes and carried data as leaves

A More Interesting Example: Expressions

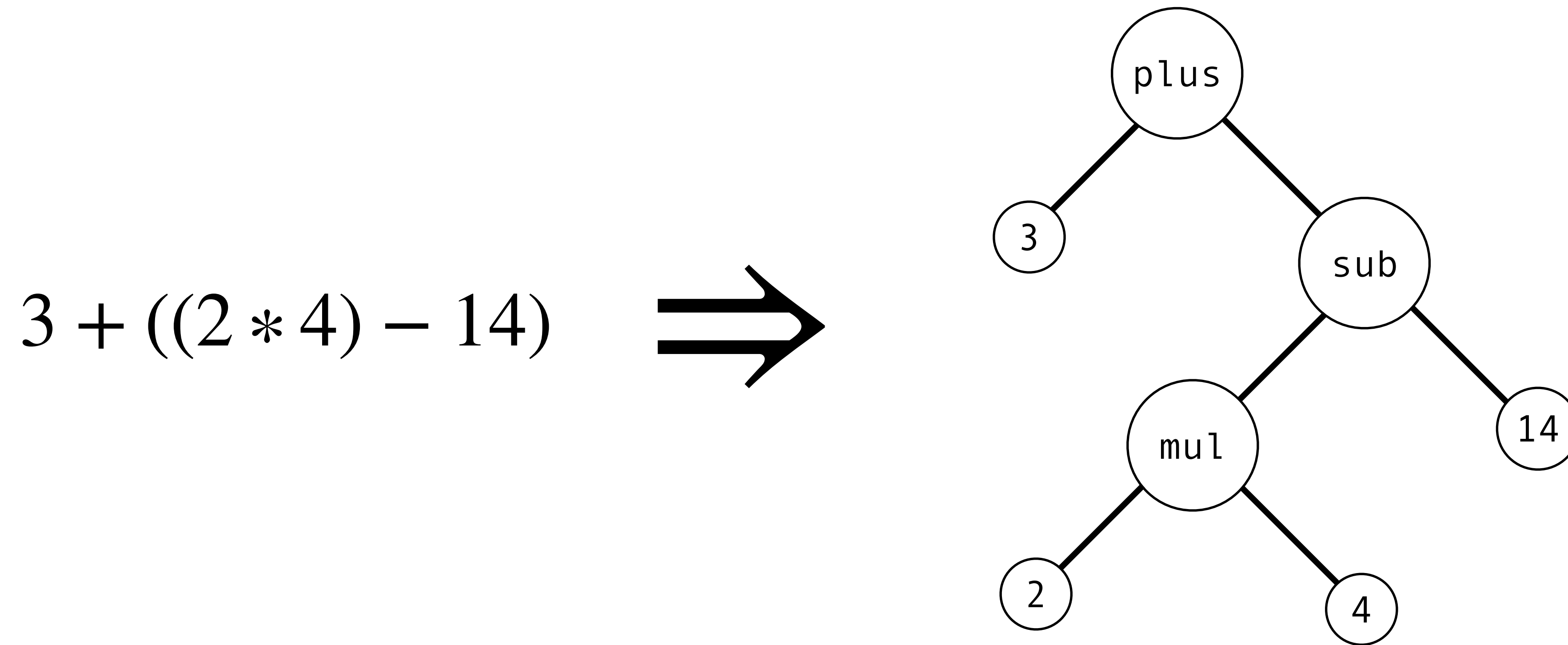
$$3 + ((2 * 4) - 14)$$

Suppose we're building a calculator...

Before we compute the value of an input, it's useful to find an **abstract representation** of the input.

This will help us separate the tasks of **evaluation** and **parsing**

A More Interesting Example: Expressions



We can represent an expression abstractly as a **tree** with operations as nodes and number values as leaves

A More Interesting Example: Expressions

```
type expr
  = Val of int
  | Add of expr * expr
  | Sub of expr * expr
  | Mul of expr * expr
```

```
let _ = Add (Val 3, Sub (Mul (Val 2, Val 4), Val 14))
```

Which means we can represent it as a recursive variant!

We 'll come back to this
in the workshop...

Parameterized Variants

```
type type variable 'a type constructor mylist  
  = Nil  
  | Cons of 'a * 'a mylist
```

```
let e1 : int mylist = Cons (1, Cons (2, Cons (3, Nil)))  
let e2 : string mylist = Cons ("1", Cons ("2", Cons ("3", Nil)))
```

The last piece of the puzzle: variants can be type agnostic

This gives us a variant which is **parametrically polymorphic**

Recall: Parametric Polymorphism

```
let rev_tail (l : 'a list) : 'a list =  
  let rec go acc l =  
    match l with  
    | [] -> acc  
    | x :: xs -> go (x :: acc) xs  
  in go [] l
```

This allows us to write functions which can be more generally applied (reversing a list **does not depend on** what's in the list)

Useful ADTs

Recall: Lists

```
type 'a list
= []
| (::) of 'a * 'a list
```

ordered unlabeled variable-length homogeneous
collections of data

Many important operations on data can be represented as operations on lists (e.g., updating all users in a database)

Options

```
type 'a option = None | Some of 'a
```

```
let head (l : 'a list) : 'a option =  
  match l with  
  | [] -> None  
  | x :: xs -> Some x
```

Options are like boxes which *may* hold a value or may be empty

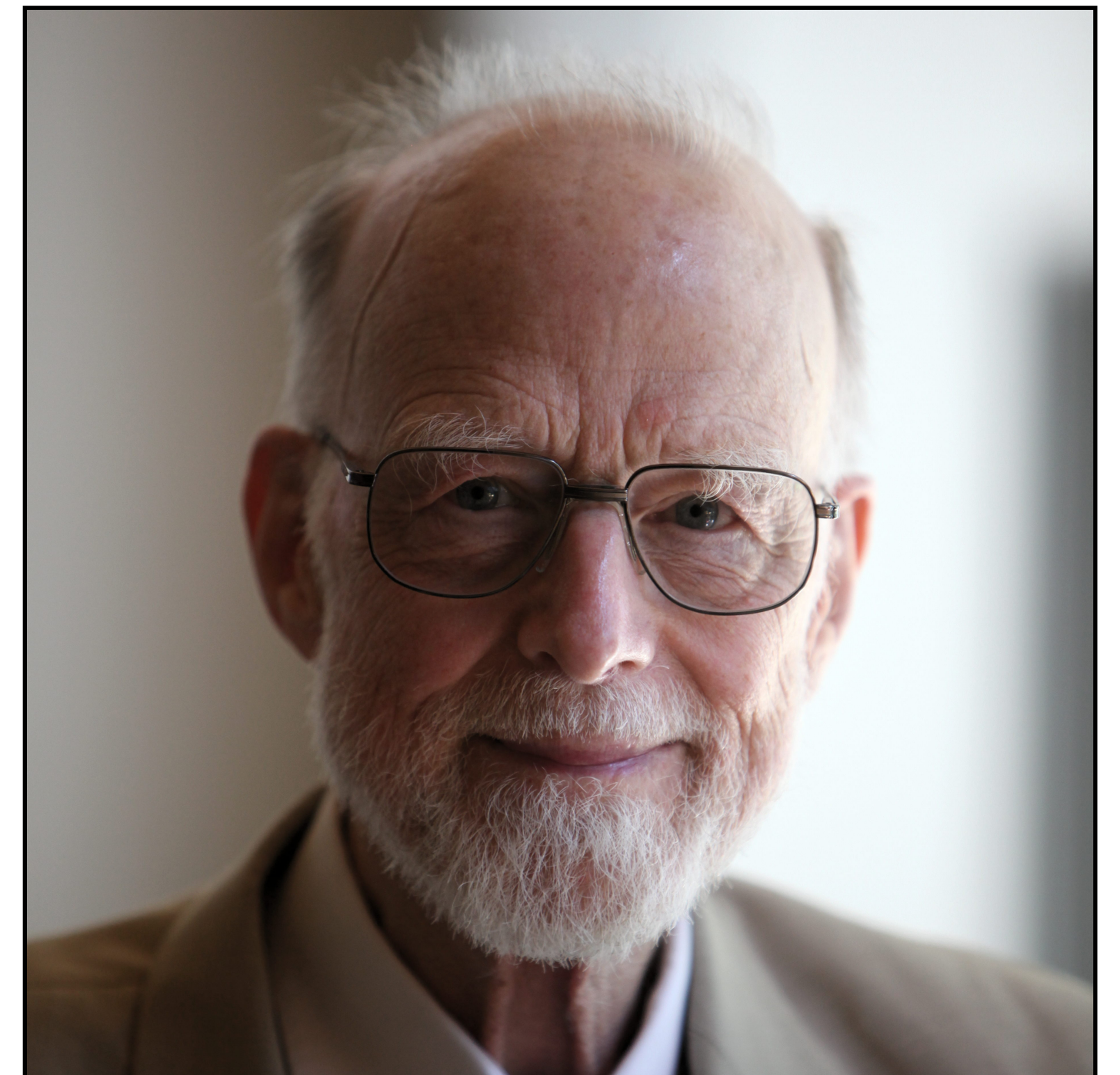
This can be useful for defining functions which **may not be total**

Aside: The Billion-Dollar Mistake

Tony Hoare calls his invention of the **null pointer** a "billion-dollar mistake"

OCaml doesn't have null pointers

I call it my billion-dollar mistake. It was the invention of the null reference in 1965. At that time, I was designing the first comprehensive type system for references in an object oriented language (ALGOL W). My goal was to ensure that all use of references should be absolutely safe, with checking performed automatically by the compiler. But I couldn't resist the temptation to put in a null reference, simply because it was so easy to implement. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years.



Results

```
type ('a, 'e) result =  
  | Ok of 'a  
  | Error of 'e  
  
let head (l : 'a list) : ('a, string) result =  
  match l with  
  | [] -> Error "[] has no first element"  
  | x :: xs -> Ok x
```

Note the syntax

Error message

A **result** is an option with additional data in the "None" case

Aside: Built-in Variants

```
utop # #show List;;  
module List :  
  sig  
    type 'a t = 'a list = [] | (::) of 'a * 'a list  
    val length : 'a t -> int  
    val compare_lengths : 'a t -> 'b t -> int  
    val compare_length_with : 'a t -> int -> int  
    val is_empty : 'a t -> bool  
    val cons : 'a -> 'a t -> 'a t  
    val hd : 'a t -> 'a  
    ...
```

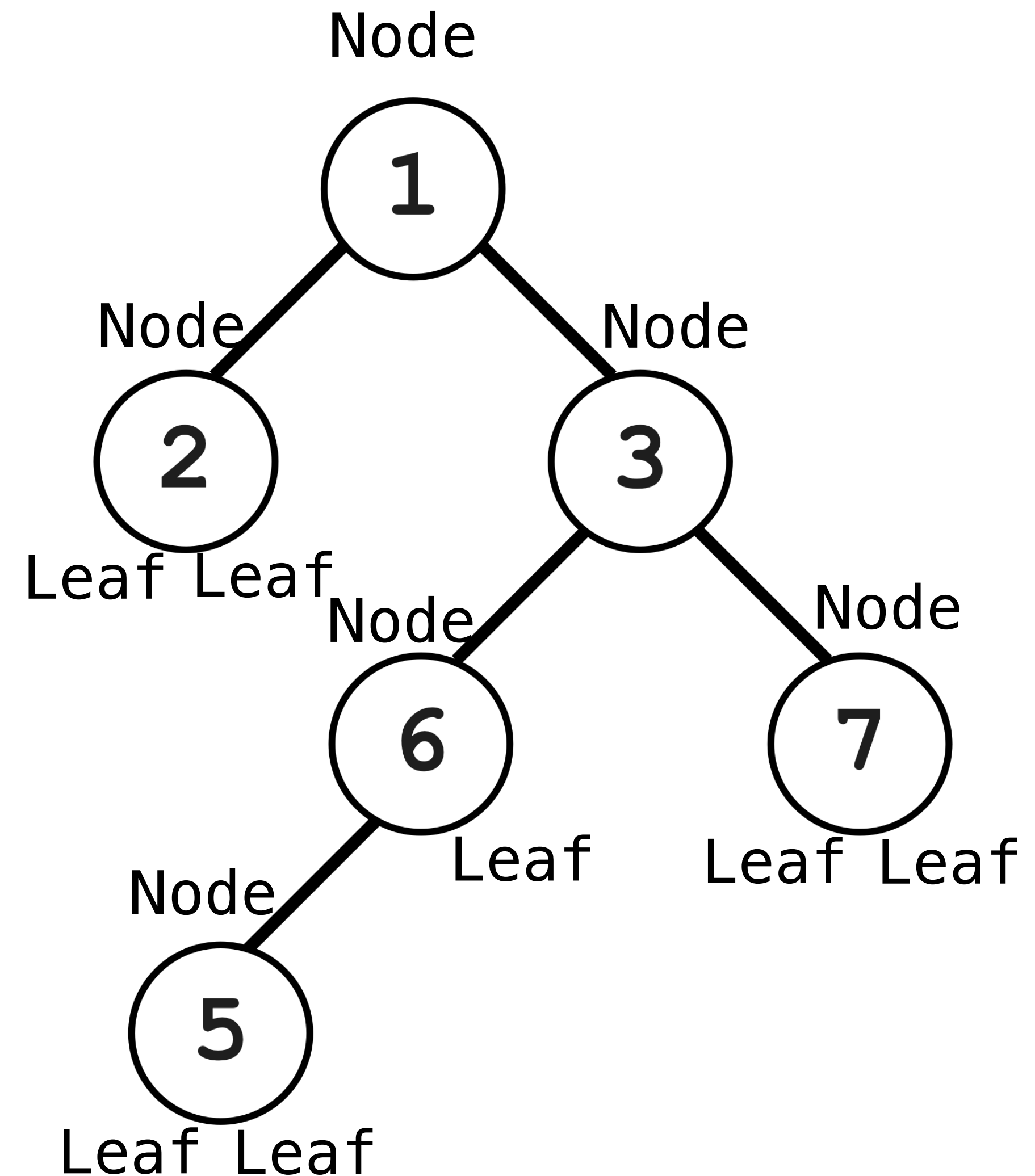
lists and optionals and results are built into OCaml

You can also use the **#show** directive in Utop to see the type signatures of functions available for lists, options and results.

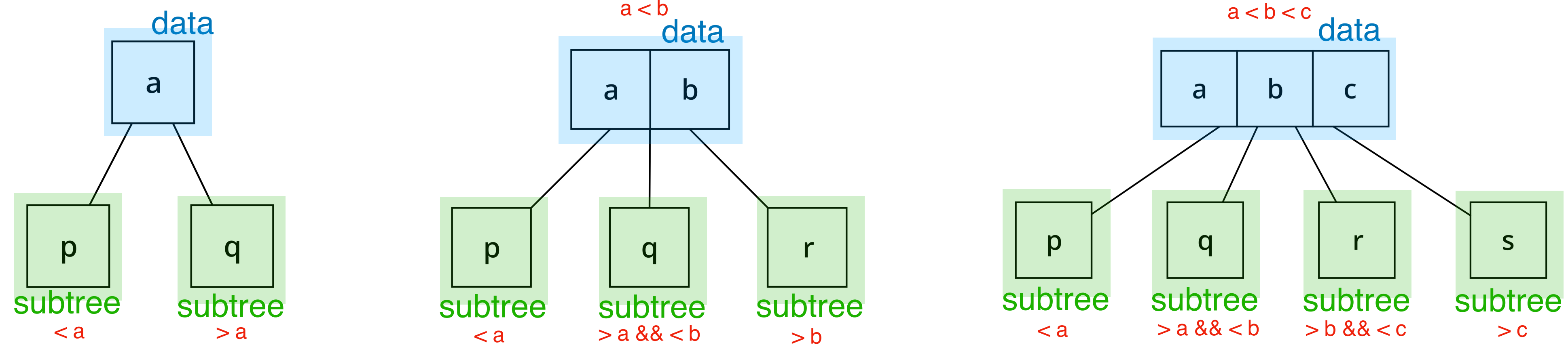
Trees

```
type 'a tree =  
  | Leaf  
  | Node of 'a * 'a tree * 'a tree
```

A tree is a leaf with a value or
a node with a left or right
subtree



Practice Problem



A **2-3-4 tree** is a self-balancing tree structure with three possible kinds of nodes, shown above. Write an ADT to represent 2-3-4 trees

(If you have extra time, try implementing search for 2-3-4 trees)

Higher-Order Programming

Concepts of Programming Languages

Outline

- » Introduce the notion of **higher-order functions** as a way to write cleaner, more general code
- » Look at three common HOFs: **map**, **filter**, **fold**

Higher-Order Functions

Higher-Order Programming

In OCaml, functions can be:

1. **returned** by another function
2. given names with **let-definitions**
3. **passed as arguments** to another function

First-Order Function Types

`int -> string`

`() -> bool`

`bool * bool -> bool`

`'a -> 'a`

Second-Order Function Types

`(int -> string) -> int -> string`

`(() -> bool) -> bool`

`'a list -> ('a -> 'b) -> 'b list`

Third-Order Functions

$(('a \rightarrow 'b) \rightarrow 'a) \rightarrow 'b$

$((() \rightarrow \text{bool}) \rightarrow \text{bool}) \rightarrow \text{bool}$

$((\text{bool} \rightarrow \text{bool}) \rightarrow \text{bool}) * \text{bool} \rightarrow \text{bool}$

And so on...

```
1st: int
2nd: int -> int
3rd: (int -> int) -> int
4th: ((int -> int) -> int) -> int
5th: (((int -> int) -> int) -> int) -> int
6th: ((((int -> int) -> int) -> int) -> int) -> int
7th: ((((((int -> int) -> int) -> int) -> int) -> int) -> int) -> int) -> int
8th: (((((((int -> int) -> int) -> int) -> int) -> int) -> int) -> int) -> int) -> int
:
```

The **higher-order** part comes from the fact that we can do this *ad infinitum*

(In practice, we rarely use higher than third-order or fourth-order functions)

The Abstraction Principle

Simple Example

```
let rec reverse (l : int list) : int list =  
  match l with  
  | [] -> []  
  | x :: xs -> reverse xs @ [x]
```

Remember that **polymorphism** allows us to write general functions by being *agnostic* about types

*It doesn't matter if we're reversing an **int list** of **string list** or an **int list list**...*

Simple Example

```
let rec fact n =  
  match n with  
  | 0 -> 1  
  | n -> n * fact (n - 1)
```

```
let rec sum n =  
  match n with  
  | 0 -> 0  
  | n -> n + sum (n - 1)
```

Some functions cannot be polymorphic

But can we still abstract the core functionality?

Simple Example

```
let rec accum f n start =  
  let rec go n =  
    match n with  
    | 0 -> start  
    | n -> f n (go (n - 1))  
  in go n
```

In order to generalize this function, we need to be able to take the *operation as a parameter*

Now we have a single function which we can *reuse* elsewhere

Another Example

```
let rec insert (x : 'a) (l : 'a list) : 'a list =  
  match l with  
  | [] -> [x]  
  | y :: ys -> if x <= y then x :: y :: ys else y :: insert x ys  
  
let rec sort (l : 'a list) : 'a list =  
  match l with  
  | [] -> []  
  | x :: xs -> insert x (sort xs)
```

Sorting *is* polymorphic

But what if we want to sort in *reverse order*, or *only on a part of the data*?

demo
(sorting)

The Abstraction Principle

The abstraction principle comes from MacLennan's **Functional Programming: Theory and Practice**

- » Abstract out core functionality
- » Use higher-order functions to parametrize by functionality specific to the problem
- » (Try to understand the algebra of programming)

Three Common HOFs

Overview

map	transform each element (keep every element)
filter	keep some elements based on a predicate
fold	combine elements via an accumulation function

Map

Definition of Map

```
let rec map f l =  
  match l with  
  | [] -> []  
  | x :: xs -> f x :: map f xs
```

Is this tail recursive?

- » *If the list is empty there is nothing to do*
- » *If the list is nonempty, we apply f to its first element, and recurse*

Tail-Recursive Map

```
let rec map_t f l =  
  let rec go l acc =  
    match l with  
    | [] -> List.rev acc  
    | x :: xs -> go xs (f x :: acc)  
  in go l []
```

For a tail-recursive version we can build the list in reverse in acc and then *reverse it at the end*

This may seem inefficient, but it's just a *constant factor* slower

Practice Problem

Implement the function

```
val pointwise_max : ('a -> int) -> ('a -> int)  
    -> 'a list -> int list
```

*so that **pointwise_max** **f g l** is **l** but with **f** or **g** applied to each element, whichever gives the larger value*

Filter

Predicates

Def. A Boolean predicate on '**a**' is a function of type '**a** -> bool'

A predicate is a function which defines a *property*

Examples:

```
let even n = n mod 2 = 0
```

```
let even_length l = even (List.length l)
```

Definition of Filter

```
let rec filter p l =  
  match l with  
  | [] -> []  
  | x :: xs ->  
    (if p x then [x] else []) @ filter p xs
```

Is this tail recursive?

- » *If the list is empty there is nothing to do*
- » *If the first element satisfies our predicate we keep it and recurse*
- » *Otherwise, we drop it and recurse*

Tail-Recursive Definition of Filter

```
let filter_tail p =  
  let rec go acc l =  
    match l with  
    | [] -> List.rev acc  
    | x :: xs -> go ((if p x then [x] else []) @ acc) xs  
  in go []
```

As with map, we have to reverse the output before returning it

Question

```
let h p q = List.filter (fun i -> p i && q i)
```

What does the above function do?

And what is the type of h?

Practice Problem

Implement the function

val negatives : int list -> int list

*so that **negatives l** is the list of negative numbers in **l**.*

Also implement the function

val gets : 'a -> ('a * 'b) list -> 'b list

*so that **gets key l** is the list of values **v** such that **(key, v)** is a member of **l***

Write a single function that can be used to implement both

Folds

A Couple Functions

```
let rec sum l =  
  match l with  
  | [] -> 0  
  | x :: xs -> x + sum xs
```

base rec. call
combine

```
let rec rev l =  
  match l with  
  | [] -> []  
  | x :: xs -> rev xs @ [x]
```

base rec. call
combine

```
let rec concat ls =  
  match ls with  
  | [] -> []  
  | xs :: xss -> xs @ concat xss
```

base rec. call
combine

```
let map f l =  
  let rec go l =  
    match l with  
    | [] -> []  
    | x :: xs -> (f x) :: go xs  
  in go l
```

base rec. call
combine

Fold as Specialized Pattern Matching

```
let rec sum l =  
  match l with  
  | [] -> 0  
  | x :: xs -> x + sum xs
```

Fold as Specialized Pattern Matching

```
let rec sum l =  
  let base = 0 in  
  match l with  
  | [] -> base  
  | x :: xs -> x + sum xs
```

Fold as Specialized Pattern Matching

```
let rec sum l =  
  let base = 0 in  
  let op = (+) in  
  match l with  
  | [] -> base  
  | x :: xs -> op x (sum xs)  
                                     fold right
```

Fold as Specialized Pattern Matching

```
let sum l =  
  let base = 0 in  
  let op = (+) in  
  List.fold_right op l base
```

Fold as Specialized Pattern Matching

```
let sum l = List.fold_right (+) l 0
```

Fold as Specialized Pattern Matching

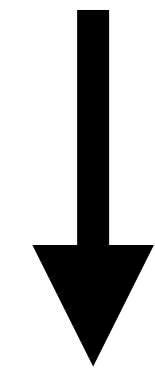
```
let sum l = List.fold_right (+) l 0
```

We get a one-liner!

The point: folds can "iterate" over a list

The Picture

1 :: (2 :: (3 :: (4 :: (5 :: (6 :: (7 :: [])))))



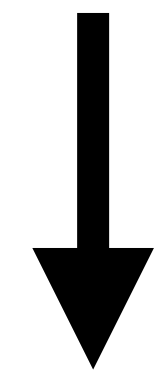
sum = fold_right (+) 1 0

1 + (2 + (3 + (4 + (5 + (6 + (7 + 0)))))

We can think of **fold_right** as "replacing" :: with + and [] with 0

The Picture

1 :: (2 :: (3 :: (4 :: (5 :: (6 :: (7 :: [])))))



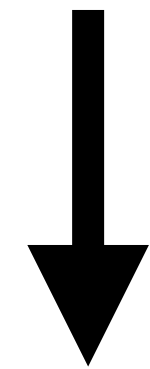
prod = fold_right (*) 1 1

1 * (2 * (3 * (4 * (5 * (6 * (7 * 1)))))

We can think of **fold_right** as "replacing" :: with * and [] with 1

The Picture

[1] :: ([2] :: ([3] :: ([4] :: ([5] :: ([6] :: ([7] :: [])))))



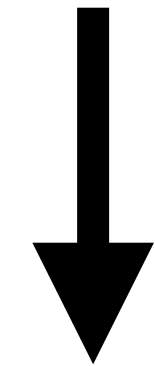
concat = fold_right (@) 1 []

[1] @ ([2] @ ([3] @ ([4] @ ([5] @ ([6] @ ([7] @ [])))))

We can think of **fold_right** as "replacing" :: with @ and [] with []

The Picture

1 :: (2 :: (3 :: (4 :: (5 :: (6 :: (7 :: []))))))



fold_right op 1 base

op 1 (op 2 (op 3 (op 4 (op 5 (op 6 (op 7 base)))))

We can think of **fold_right** as "replacing" :: with op and [] with base

Definition of Fold Right

note the order of args.

```
let fold_right op l base =  
  let rec go l =  
    match l with  
    | [] -> base  
    | x :: xs -> op x (go xs)  
  in go l
```

Is this tail recursive?

- » On empty, return the **base** element
- » On nonempty, recurse on the tail and apply **op** to the head and the result

Practice Problem

Write `filter` using `List.fold_right`

Write `append (@)` using `List.fold_right`

Definition of Fold Left

note the order of args.

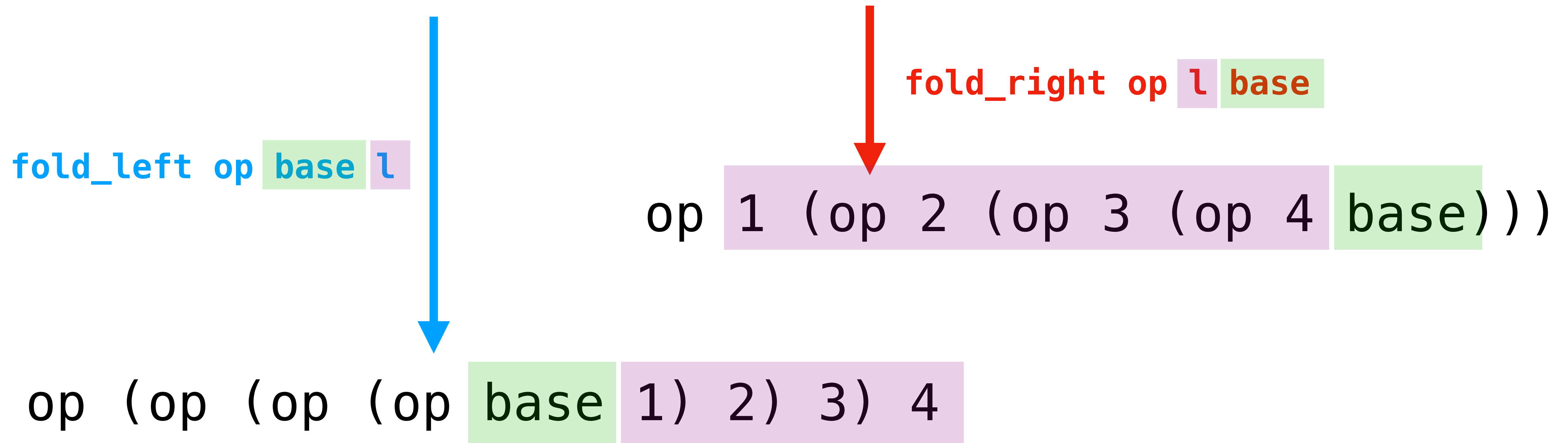
```
let fold_left op base l =  
  let rec go l acc =  
    match l with  
    | [] -> acc  
    | x :: xs -> go xs (op acc x)  
  in go l base
```

Folding left is just our incorrect tail recursive right folding (with a change in the order of arguments)

fold_left is a **left**-associative fold
fold_right is a **right**-associative fold

The Picture

1 :: (2 :: (3 :: (4 :: [])))



Tail-Recursive Fold Right

```
let fold_right_tr op l base =  
  List.fold_left  
    (fun x y -> op y x)  
    base  
    (List.rev l)
```

We can write fold_right in terms of fold left by reversing the list and "reversing" the operation

Challenge: Write a tail-recursive fold right without reversing the list

Short Circuiting

```
let rec all bs =  
  match bs with  
  | [] -> true  
  | false :: _ -> false  
  | true :: t -> all t
```

```
let all = List.fold_left (&&) true
```

Which is better?

fold_left has to traverse the entire list, it can't short-circuit

But the fold code is shorter and arguably clearer...

Associativity

Def. A binary operation $\square: A \times A \rightarrow A$ is **associative** if it satisfies

$$a \square (b \square c) = (a \square b) \square c$$

for any $a, b, c \in A$

Ex. Addition and multiplication are associative, whereas subtraction and division are not

General Rules for Folds

- » For **associative** operations, use **fold_left**
- » The types are difficult to remember, let the compiler remind you
- » Don't use folds for everything, but also don't use pattern matching for everything. *Think about the use case*

Practice Problem

```
let rec insert le v l =  
  match l with  
  | [] -> [v]  
  | x :: xs ->  
    if le v x  
    then v :: l  
    else x :: insert le v xs
```

In terms of **fold_left** implement the function

val sort : ('a -> 'a -> bool) -> 'a list -> 'a list

so that **sort le l** is the list **l** in sorted order according to **le**

Summary

Higher-order function allow for better **abstraction** because we can **parameterize** functions by other functions

map and **filter** and **fold** very common patterns which can be used to write clean and simple code